



Introduction to Scientific Computation

Lecture 0

Fall 2026

The rules of the game
Intro

General course info

Lectures can be downloaded and viewed on Github:

<https://github.com/Aelphy/ISC>

There you can find information on how to prepare the environment, launch the code, submit assignments, communicate with staff etc.

Team & TAs:

Mikhail Usvyatsov

Yury Kartynnik

Communication system

Communication with staff (questions, suggestions, etc) could be done via emails m.usvyatsov+ethz@gmail.com.

Please make sure that the subject in your email starts with [ISC2026].

A Telegram chatroom for students is here: <https://t.me/iscfall26>

Plagiarism

Students are encouraged to discuss the material, but all the assignments should be done individually. Assignments will be checked for plagiarism. If noticed, it can be punished in various ways from zeroing out some parts of a specific assignment to an unsatisfactory grade for the course.

Assignments

This course uses Github for submitting assignments. You should create a dedicated **private!** repository with the name **ISC2026-solutions**. E.g. github.com/YOUR_NAME/ISC2026-solutions. And add us (@Aelphy, @kartynnik) as collaborators.

An assignment becomes submitted after sending an email to m.usvyatsov+ethz@gmail.com with the subject [ISC2026: assignment_N], where N is the number of the assignment. In the body of this email there should be a link to a dedicated directory for this particular assignment. For the first assignment it will be: github.com/YOUR_NAME/ISC2026-solutions/assignment_0

LLMs

Various AI systems became way too advanced as of Fall 2026.

When extensive similarity with AI generated content is suspected, the student claiming the authorship will face a defence procedure: detailed questions testing how well the student understood the underlying matters.

Deadlines & Attendance

Unless stated otherwise, submission deadline is 00:01 of the 10's day after handout. You lose 10% points per day until submission (from the full amount of points on the previous day).

By the time of grading, the assignment will be checked in the state of the last commit before the deadline. All later commits are subject to late submission policy.

Assignment can't yield negative amount of points, even if submitted 2 years past deadline.

Attendance is not strict, but do not disappoint us.

Grading

The course grade depends entirely on points obtained mostly by completing homework assignments. There will be no course exam at the end.

Some assignment examples are "Implement and optimise factorial computation algorithm", "Implement gradient descent algorithm for logistic regression", "Compare Ridge and Lasso regression for house prices problem", "Implement Perceptron Algorithm"

Materials

The Hitchhiker's Guide to Python

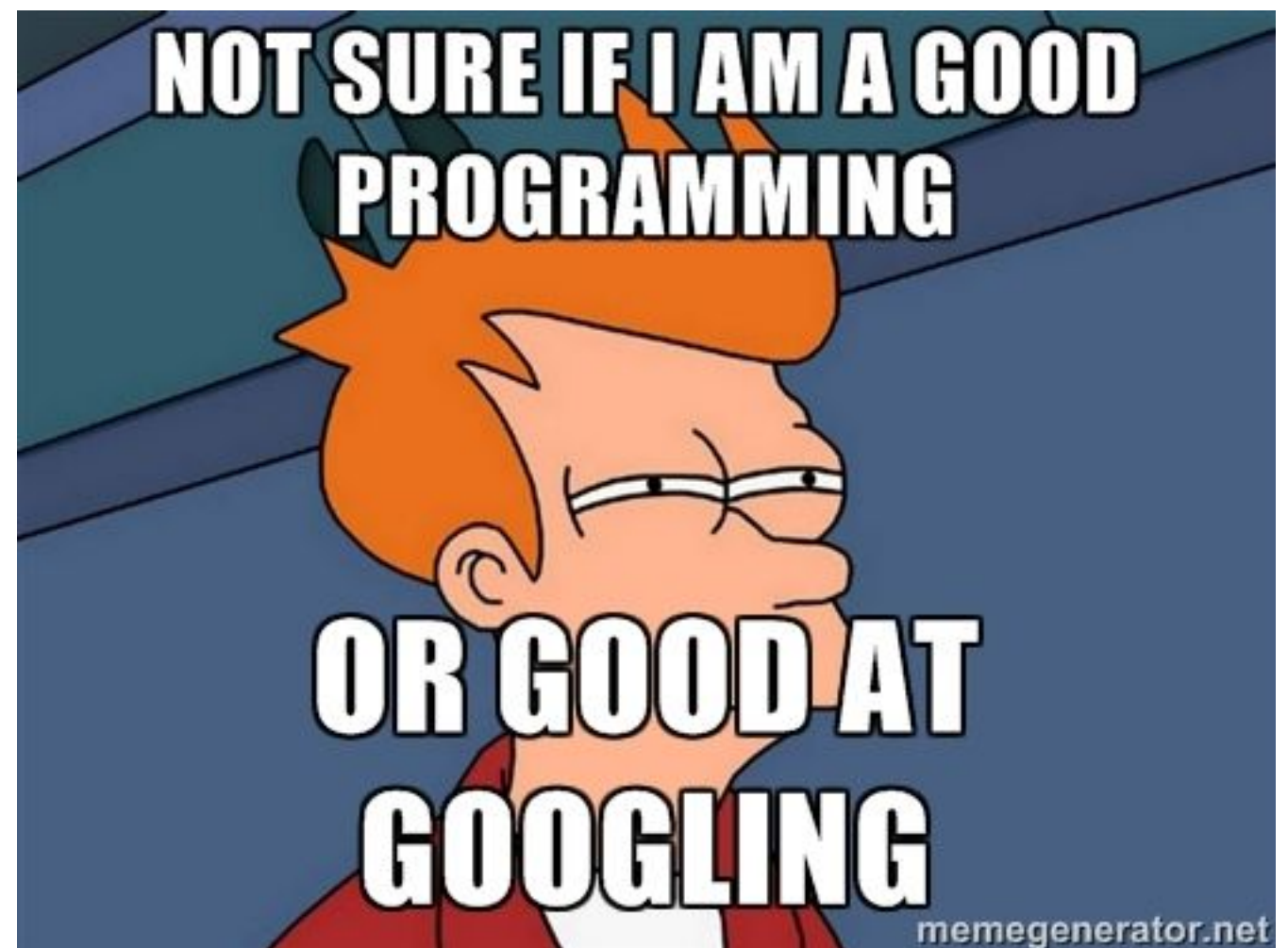
A Primer on Scientific Programming With Python. H. Langtangen

Always use

Google

StackOverflow

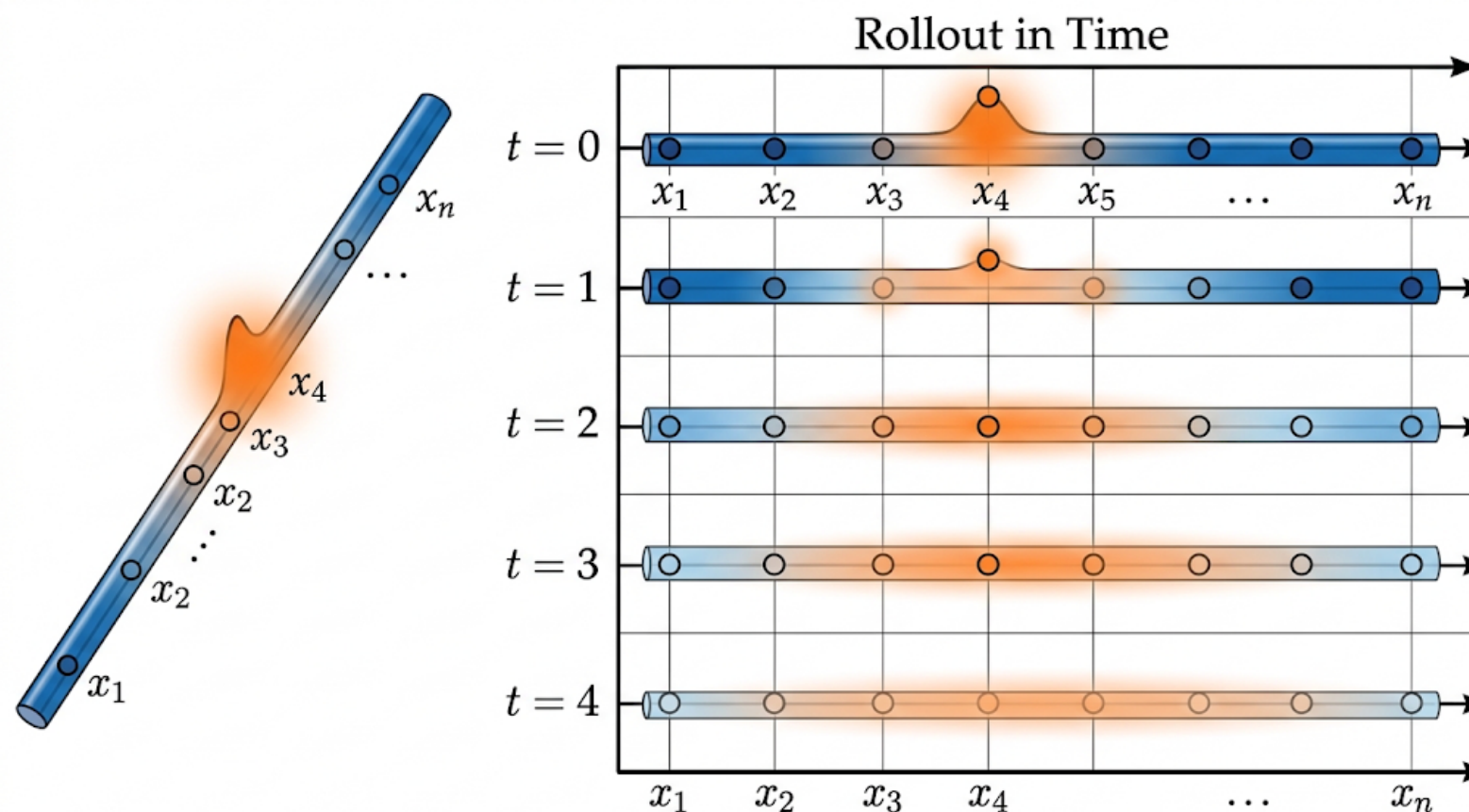
As of Fall 2026,
ChatGPT, Claude, Gemini, Grok, etc.,
might or might not be a good
resource to refer to. Mind
the references.



Introduction: What is scientific computing

Scientific computing is an applied discipline concerned with *numerical simulations* of natural and social science phenomena.

When we can describe the laws governing some phenomenon, but don't have an analytical, closed-form description of its desired characteristics, we can attempt to *discretize* it and *approximate* its numerical parameters at the resulting discrete points.

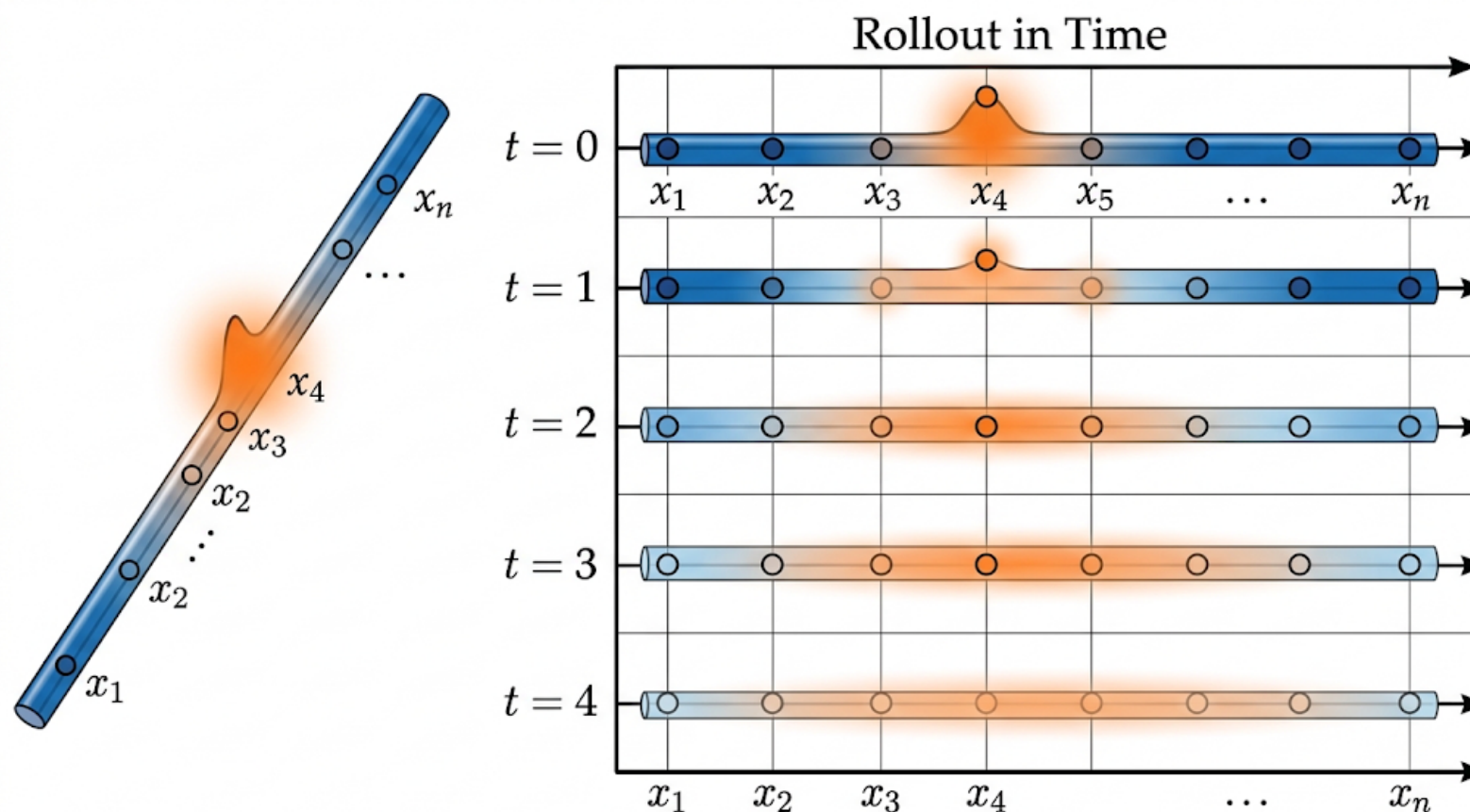


Two primary sources of approximation for tractability

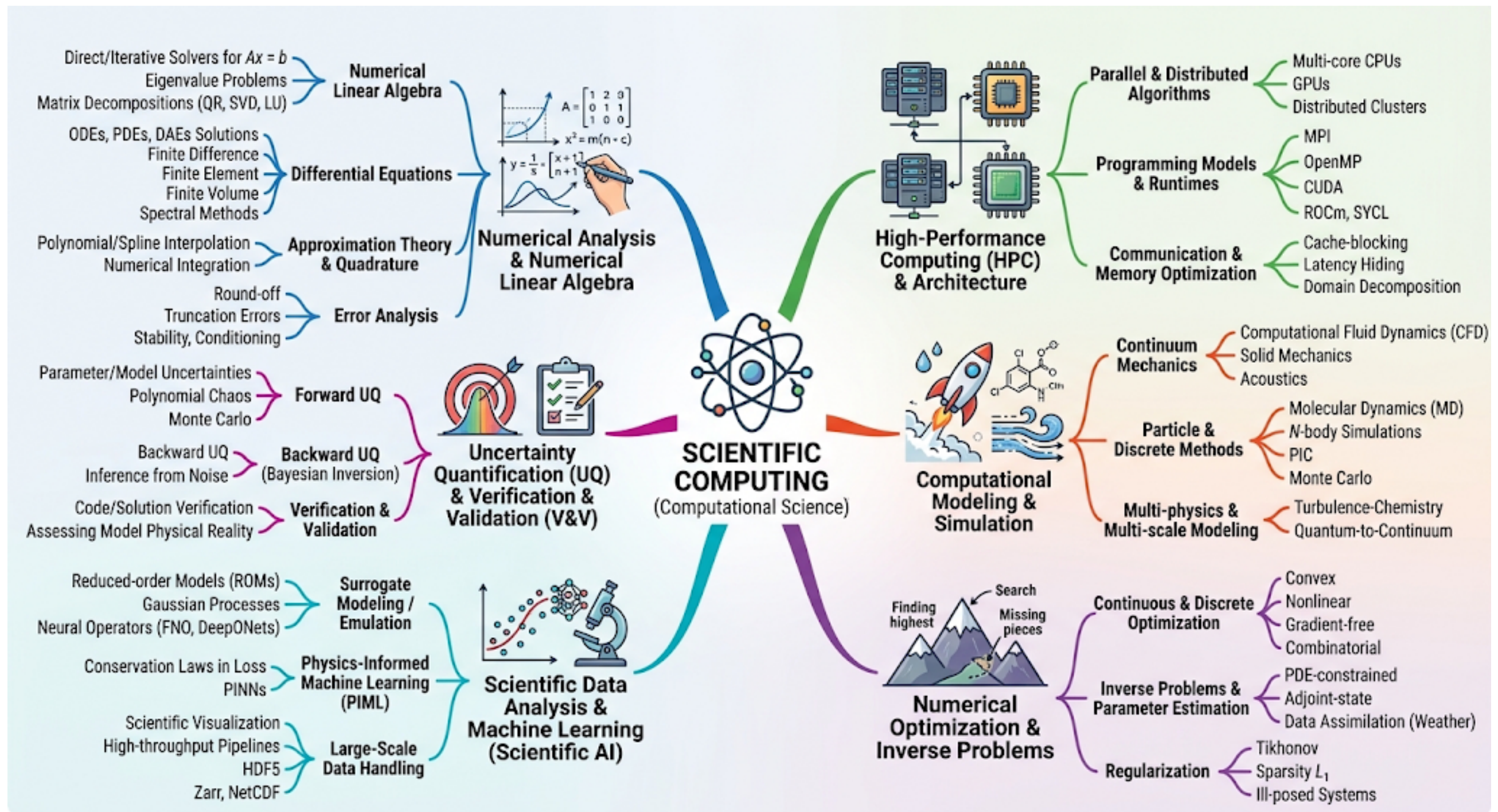
Continuous time and space are *sampled* at a finite number of points.

Continuous values are approximated, typically via rounded base-2 scientific notation (*floating point numbers*).

Numerical stability of a computational method is its level of resilience to the errors arising from the above simplifications.



Branches of scientific computing



Practical Introduction

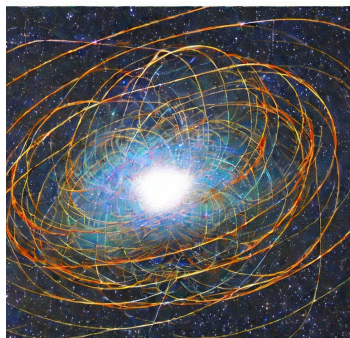
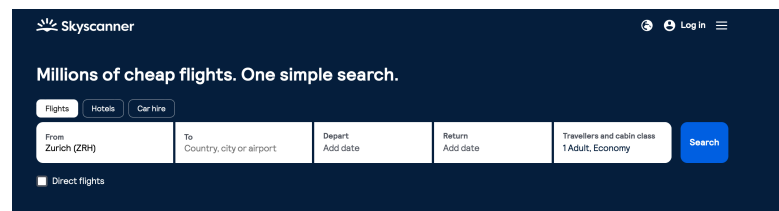
How is your code running?



Semantic gaps

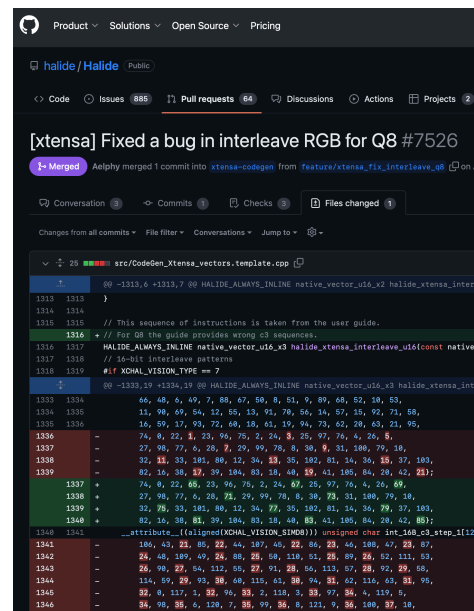
Domain applications

- Tickets search platform
 - Airlines
 - Routes
 - Prices
- Modelling celestial motion
 - Speed
 - Space
 - Mass
 - Coordinates
 - Time



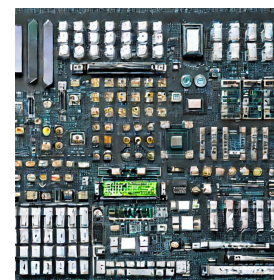
Programming primitives

- Class
- Function
- Variable
- Data structures
 - Array
 - List



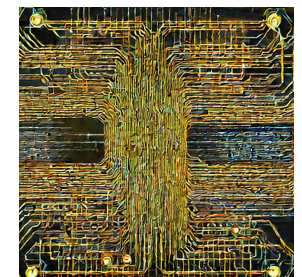
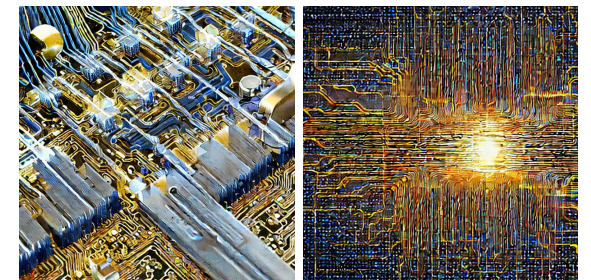
Hardware

- CPU
 - ARM
 - X86
 - RISC-V
 - Instruction set
 - ASIC
- GPU
- Bytes
- Memory
- Registers
- Cache
- RAM
- Bus



Physics

- Transistors
 - Atoms
 - Electrons
- Semiconductors
 - Lithography
 - Nanometers
- Heat
- Radio waves
- Superconductors



Images were generated with Stable Diffusion from HuggingFace, unless the source is present on an image

Semantic gaps

Programming primitives

- Class
- Function
- Variable
- Data structures
 - Array
 - List



Hardware

- CPU
 - ARM
 - X86
 - RISC-V
 - Instruction set
- GPU
- Bytes
- Memory
- Registers
- Cache
- RAM
- Bus

So, the main compiler's task is to map human-readable code to hardware to solve the problem efficiently

Machine Code & Assembly Language

Off-topic

Mnemonic notation:

ADD 5 7



ADD i j

Binary code of the instruction:

1101010010100111



Hexadecimal code of the instruction:

D4A7

Assembler code:

.format 32

R5 += R7



- The ADD instruction denotes the two's complement arithmetic addition. The contents of registers Ri and Rj are arithmetically added, and the result is put into the register Rj.
- The instruction format is as follows:



- Memory state is not considered in the instruction, and the memory state does not change.
- If the addition gives a result which cannot be put into the format specified in the instruction, then **overflow** happens

Suggested assembly statement for the ADD instruction:

Rj += Ri;

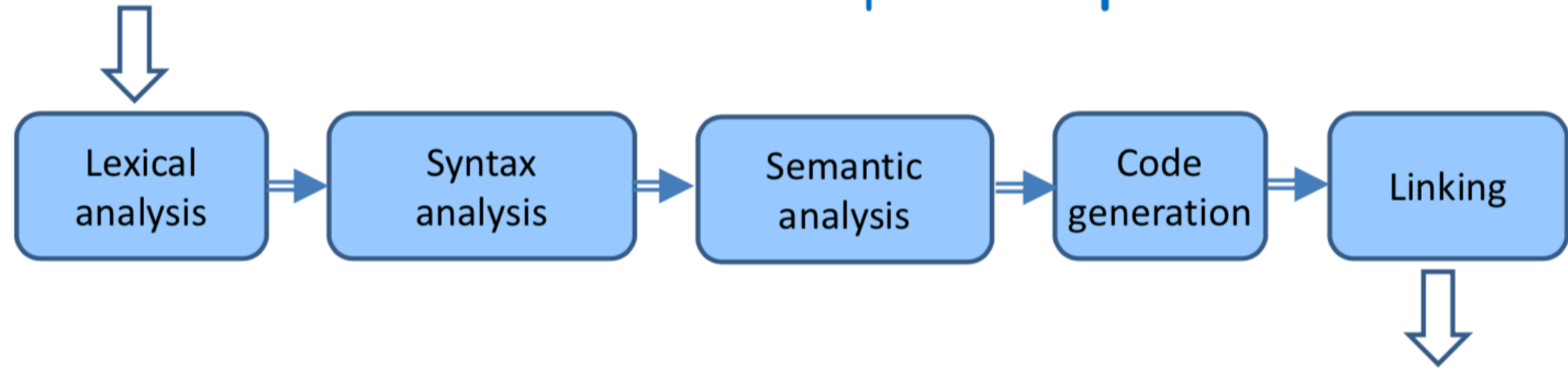
Additional assembly directives specifying the current instruction format:

.format 8; or .format 16; or .format 32;

Compilation: An Ideal Picture

*A program written by a human
(or by another program)*

Source program
text



Blue squares just denote some **actions** typical to any compiler; they are not necessarily actual compiler **components**.

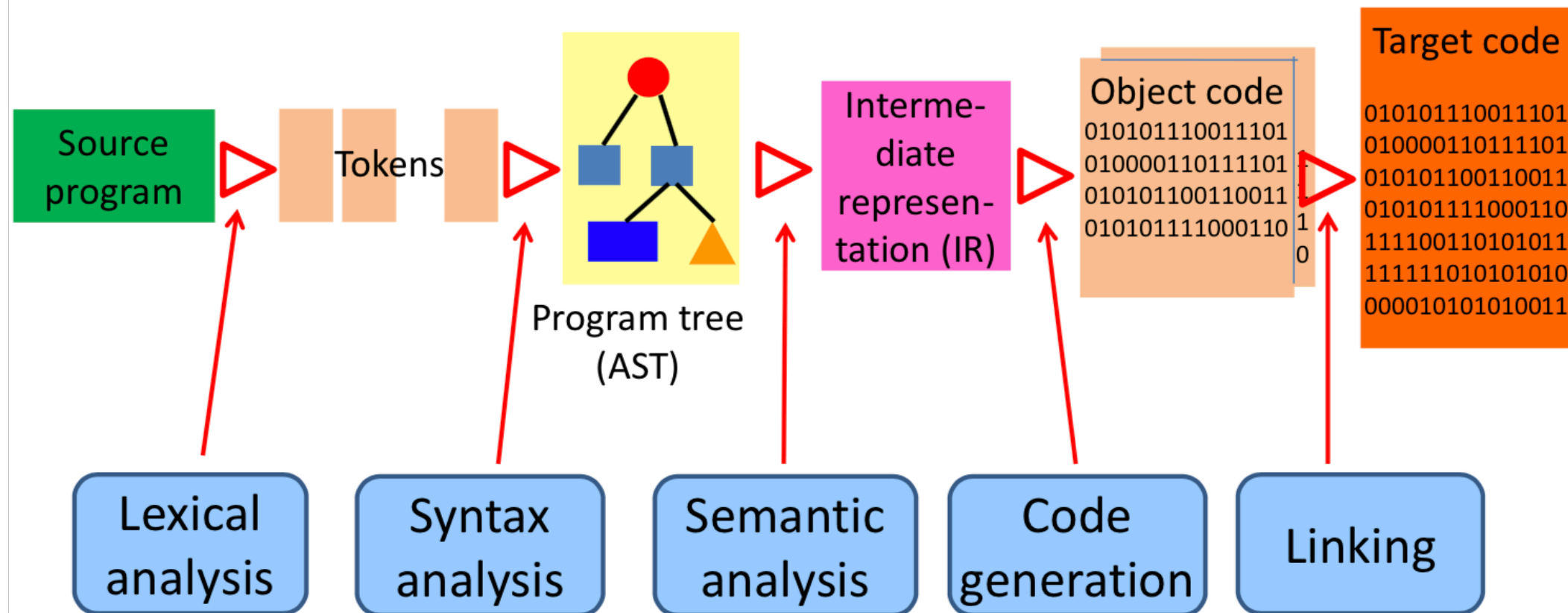
Target code

```
01010111001110101000  
01101111010101011001  
10011010101111000110
```

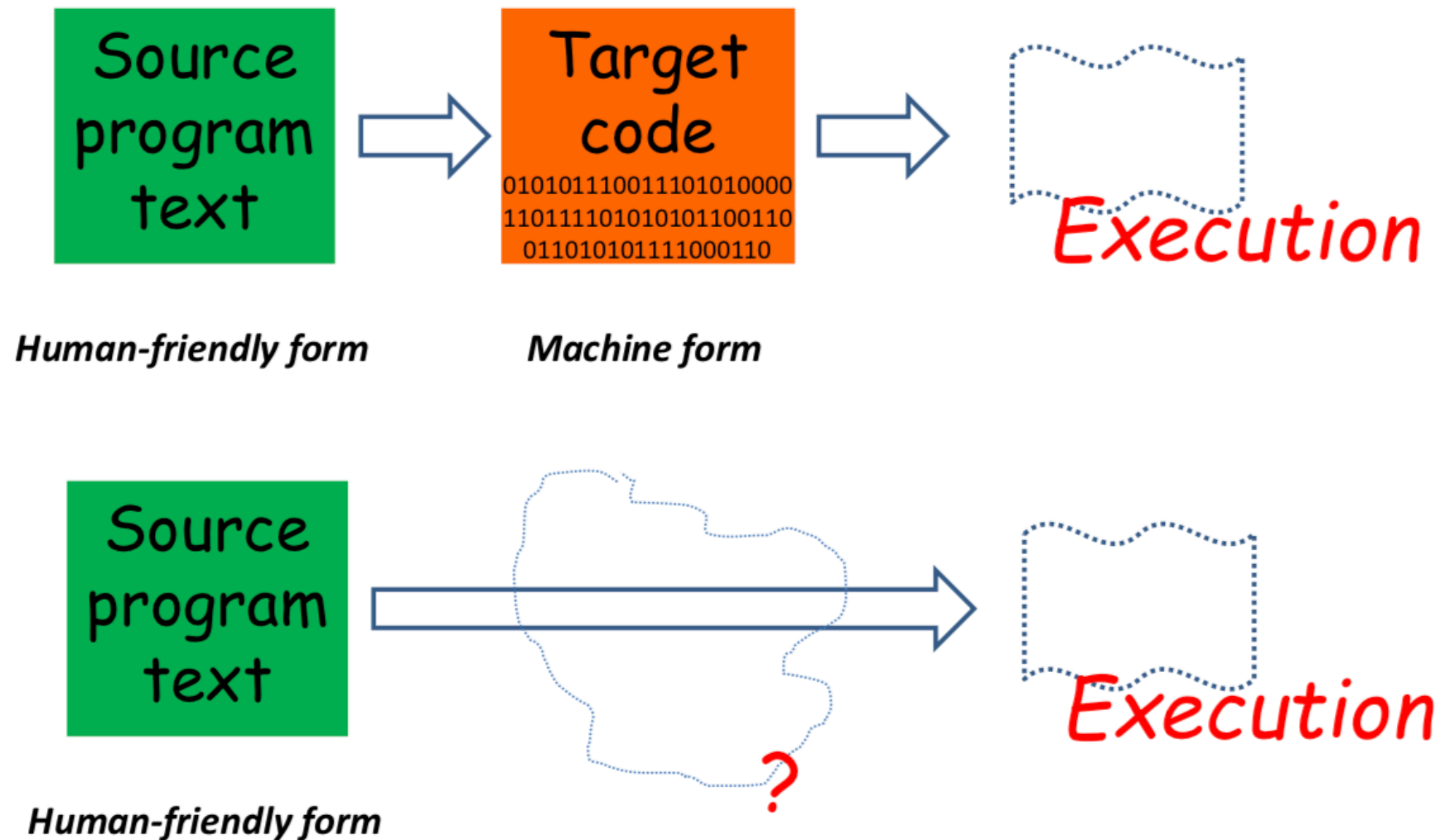
*A program binary image
suitable for immediate
execution by a machine*

Compilation Data Structures

This is a different, **data-centric** view at the compilation process. Compilation is a **sequence of transformations** of a source program.



Compilation vs Interpretation



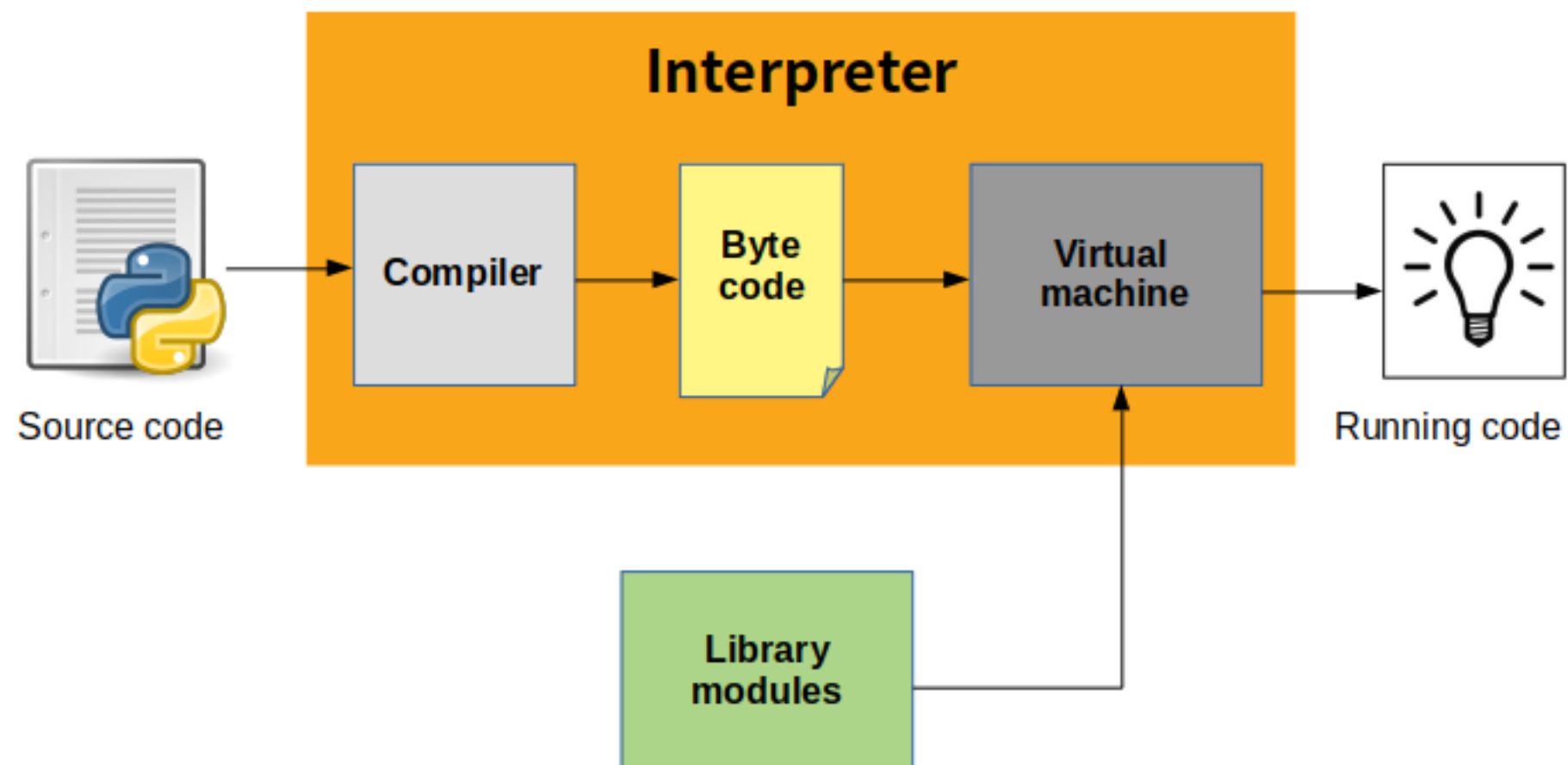
(c) Copyright 2017, E.Zouev

So, what about Python ?

From a programmer's point of view



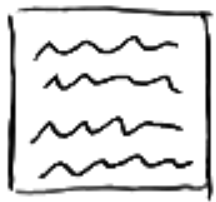
From Python's point of view



So, what about Python ?

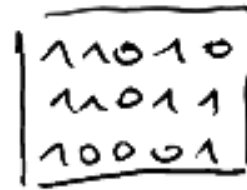
Source code:

hello.c



→ COMPILER →

Machine code:



Program (also
called binary,
executable ...)

run the
program

result



Source code:

hello.py



→ INTERPRETER → result



How does Python interpret this:

$$X + Y * Z + U$$

How does Python interpret this:

$$X + Y * Z + U$$


Stack

```
push x  
push y  
push z  
multiply  
add  
push u  
add
```

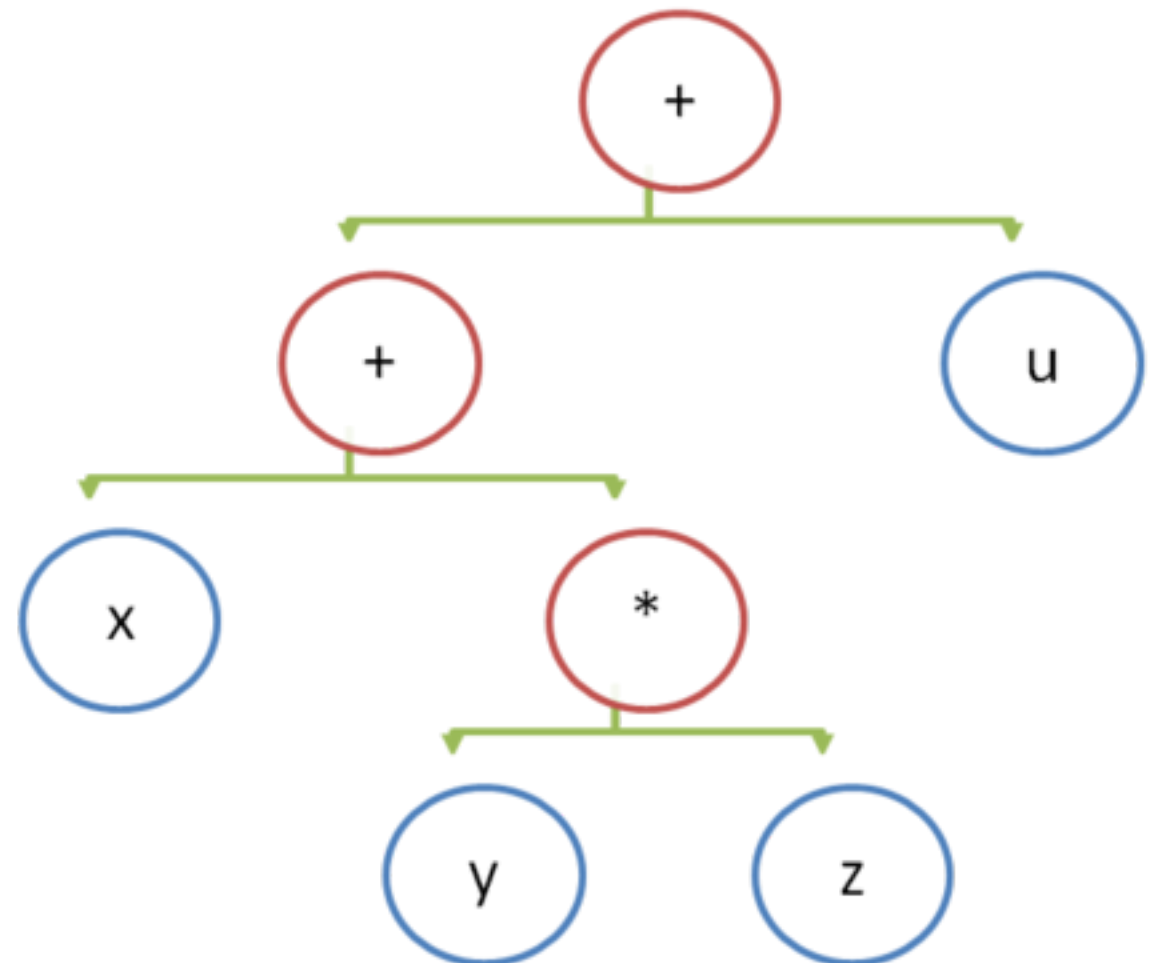
How does Python interpret this:

Stack

```
push x  
push y  
push z  
multiply  
add  
push u  
add
```

~

AST



Python

Pros:

- **Extensive Support Libraries**
- **Can use C/C++ libs via bindings**
- **Cross-platform**
- **Relatively low entry level**
- **Free & open-source**
- **Huge community**

Cons:

- **Slower than C/C++, assembly**
- **Mostly dynamic typing: runtime errors in places where static languages don't compile**
- **Parallelization gaps (Global Interpreter Lock until recently)**

Python environments

pip - Pip Installs Packages - package manager for Python.
Can be used together with conda but very carefully

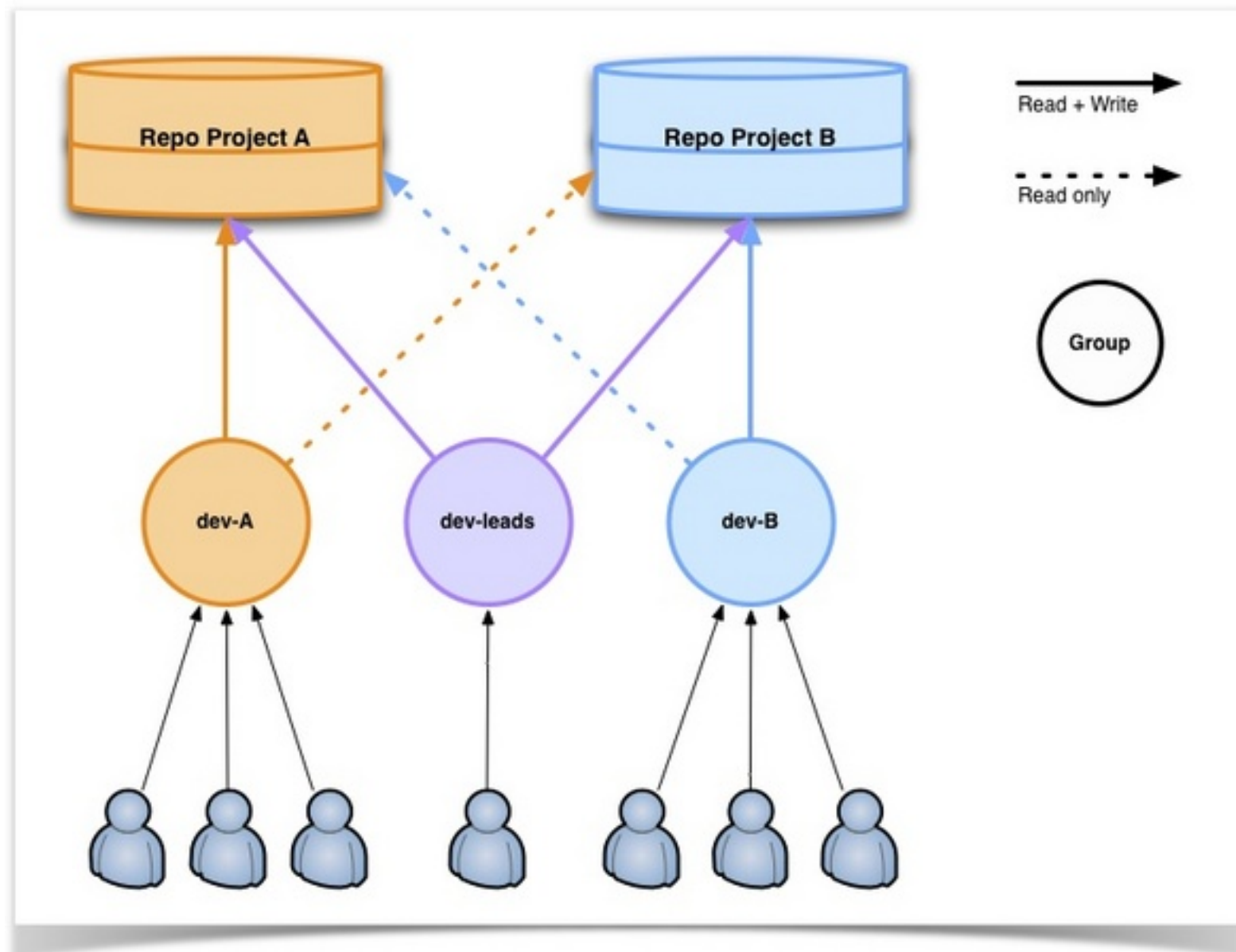
If you use python without anaconda:

- virtualenv, source activate, pip install

If you use anaconda:

- conda activate, conda install + pip install

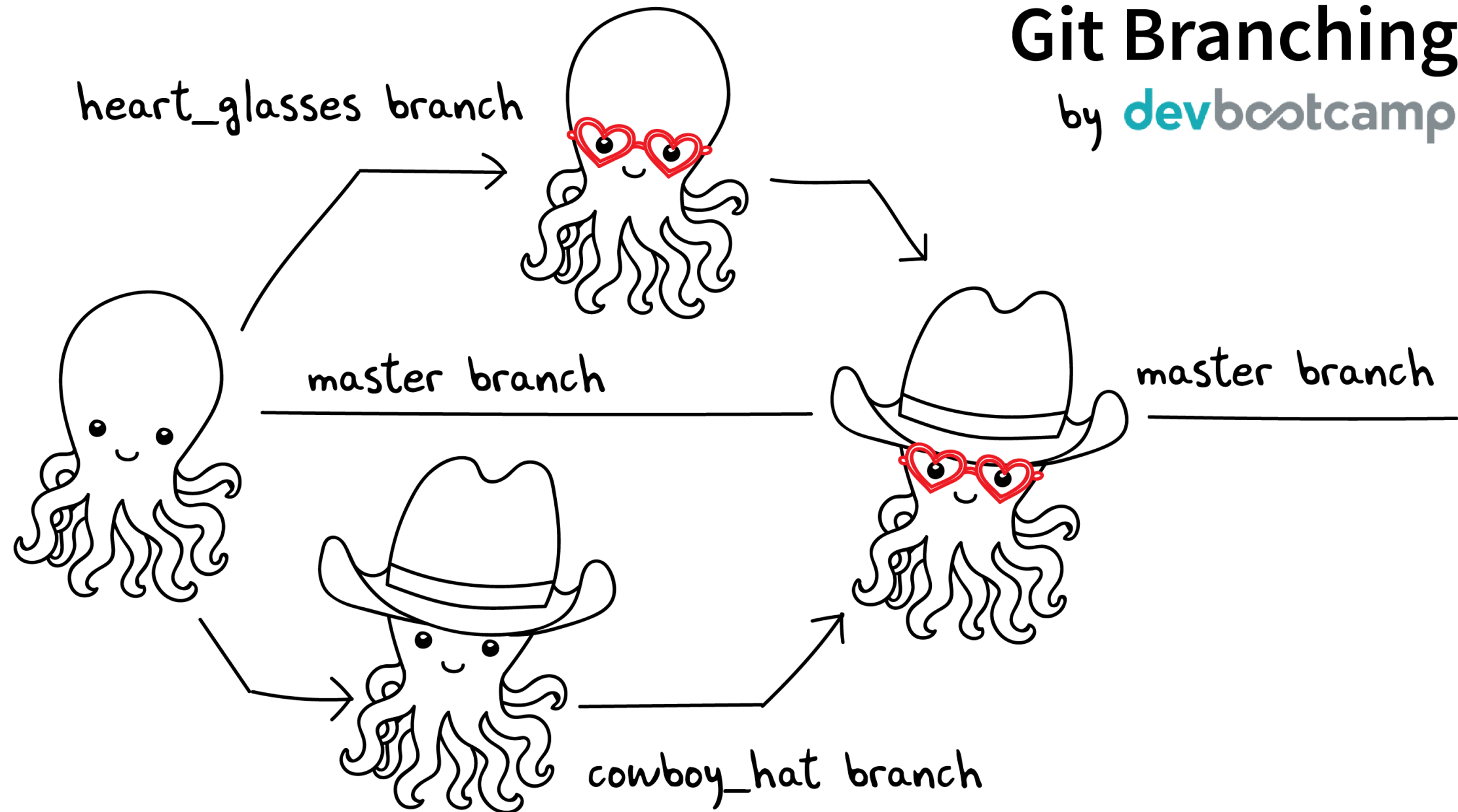
Git: repositories



Git: branches

Git Branching

by [devbootcamp](#)



Let's code

